Student Name: Ranjeet Kumar Sharma

Batch: June

Course: Data Analytics with AI

Assignment: Advanced SQL

Submission Date:- 11-08-2026

Email_id: Ranjeet Kumar (Sharma)

➔ **Write the Question and Answers.**

Q1. What is a Common Table Expression (CTE), and how does it improve SQL query readability?

Ans:- A CTE (Common Table Expression) is a temporary named result set that is created using the `WITH` keyword and can be used within a SQL query.

Q2. Why are some views updatable while others are read-only? Explain with an example.

Ans:-Some views are updatable because they directly represent rows from a single table. Other views are read-only because they contain operations such as `GROUP BY`, aggregate functions like `SUM()` or `AVG()`, or `DISTINCT`, which make direct updates difficult or impossible.

Q3. What advantages do stored procedures offer compared to writing raw SQL queries repeatedly?

Ans:- Stored procedures allow us to save SQL statements and reuse them whenever needed. They reduce code repetition, make complex operations easier to manage, improve consistency, and can accept parameters to work with different values.

Q4. What is the purpose of triggers in a database? Mention one use case where a trigger is essential.

Ans:- A trigger is used to automatically execute a predefined action when a specific database events occur, such as `INSERT`, `UPDATE`, or `DELETE`.

Use case:

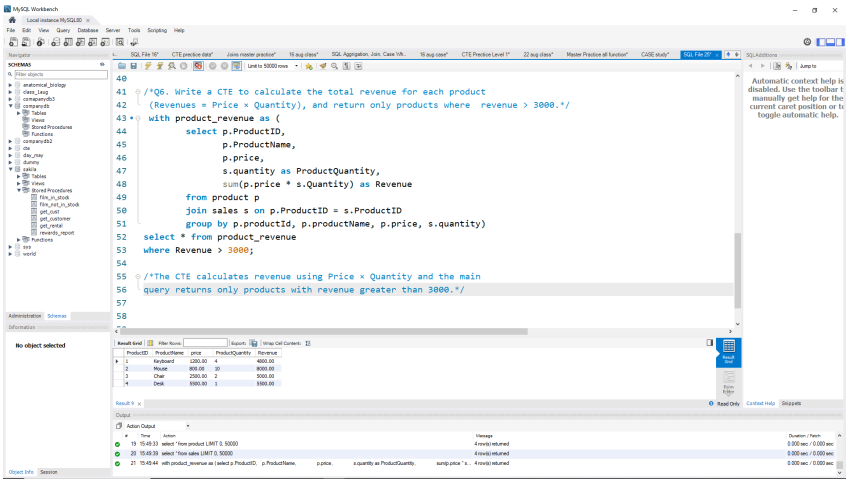
A trigger can automatically archive a product's information into an archive table whenever that product is deleted.

Q5. Explain the need for data modelling and normalization when designing a database.

Ans:- Data modelling is used to design the structure of a database, including tables, columns, and relationships between tables. Normalization is used to organize data properly, reduce data redundancy, and improve data consistency and integrity.

Q6. Write a CTE to calculate the total revenue for each product (Revenues = Price × Quantity), and return only products where revenue > 3000.

Ans:- The CTE calculates revenue using Price × Quantity and the main query returns only products with revenue greater than 3000.

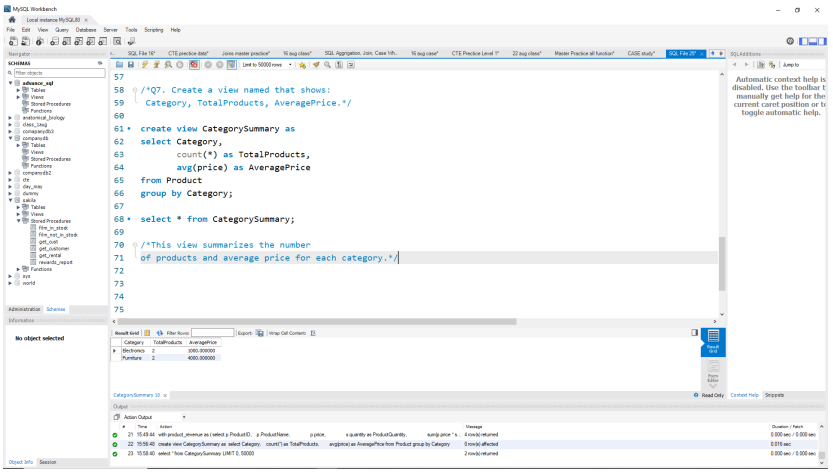


```sql
/*Q6. Write a CTE to calculate the total revenue for each product
(Revenues = Price × Quantity), and return only products where revenue > 3000.*/
with product_revenue as (
        select p.ProductID,
               p.ProductName,
               p.price,
               s.quantity as ProductQuantity,
               sum(p.price * s.Quantity) as Revenue
        from product p
        join sales s on p.ProductID = s.ProductID
        group by p.productId, p.productName, p.price, s.quantity)
select * from product_revenue
where Revenue > 3000;

/*The CTE calculates revenue using Price × Quantity and the main
query returns only products with revenue greater than 3000.*/
```

Q7. Create a view named that shows: Category, TotalProducts, AveragePrice.

Ans:- This view summarizes the number of products and average price for each category.



```sql
/*Q7. Create a view named that shows:
Category, TotalProducts, AveragePrice.*/

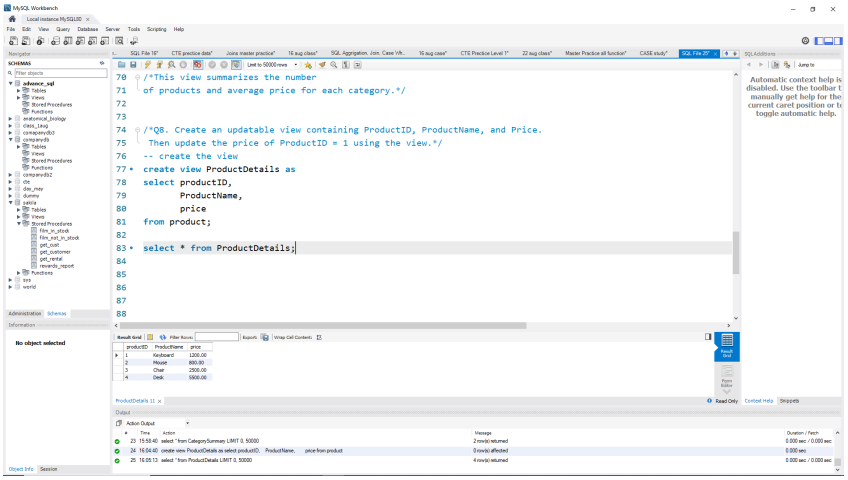
create view CategorySummary as
select Category,
       count(*) as TotalProducts,
       avg(price) as AveragePrice
from Product
group by Category;

select * from CategorySummary;

/*This view summarizes the number
of products and average price for each category.*/
```

Q8. Create an updatable view containing ProductID, ProductName, and Price. Then update the price of ProductID = 1 using the view.

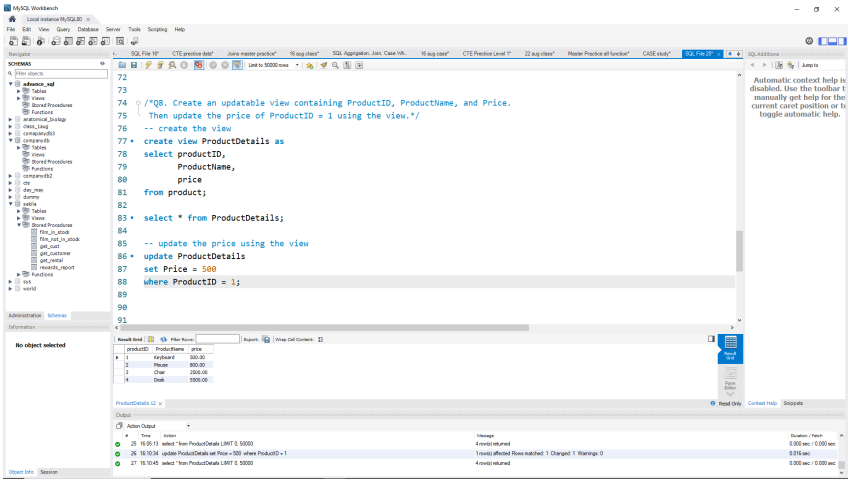
Ans:- Step 1:- Create the view



```
/*This view summarizes the number
of products and average price for each category.*/

/*Q8. Create an updatable view containing ProductID, ProductName, and Price.
 Then update the price of ProductID = 1 using the view.*/
-- create the view
create view ProductDetails as
select productID,
       ProductName,
       price
from product;
select * from ProductDetails;
```

Step 2:- update the price using the view



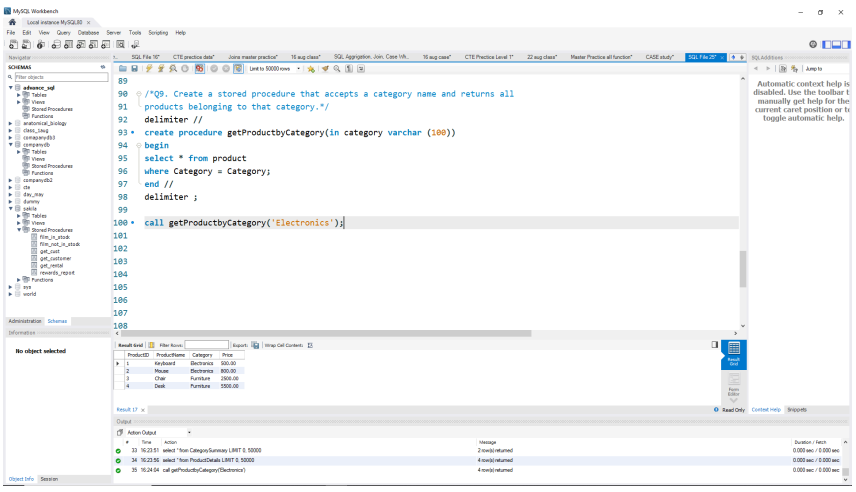
```
/*Q8. Create an updatable view containing ProductID, ProductName, and Price.
 Then update the price of ProductID = 1 using the view.*/
-- create the view
create view ProductDetails as
select productID,
       ProductName,
       price
from product;

select * from ProductDetails;

-- update the price using the view
update ProductDetails
set Price = 500
where ProductID = 1;
```

Q9. Create a stored procedure that accepts a category name and returns all products belonging to that category.

Ans:-The procedure accepts a category name as an input parameter and returns all products belonging to that category.

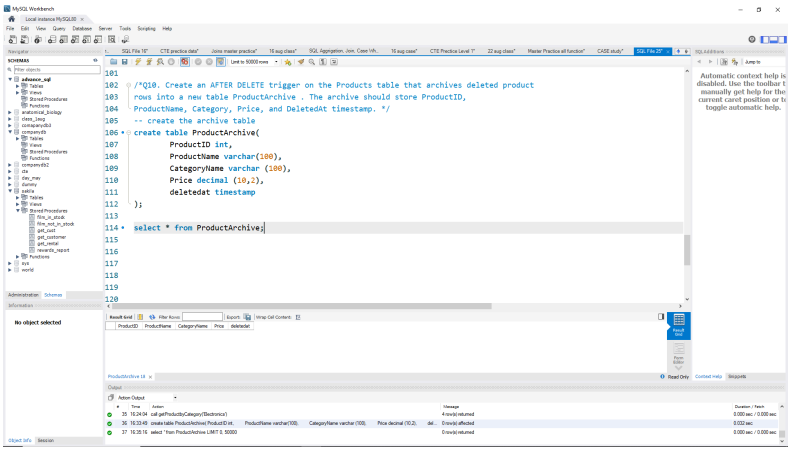


```
/*Q9. Create a stored procedure that accepts a category name and returns all
products belonging to that category.*/
delimiter //
create procedure getProductbyCategory(in category varchar (100))
begin
select * from product
where Category = Category;
end //
delimiter ;

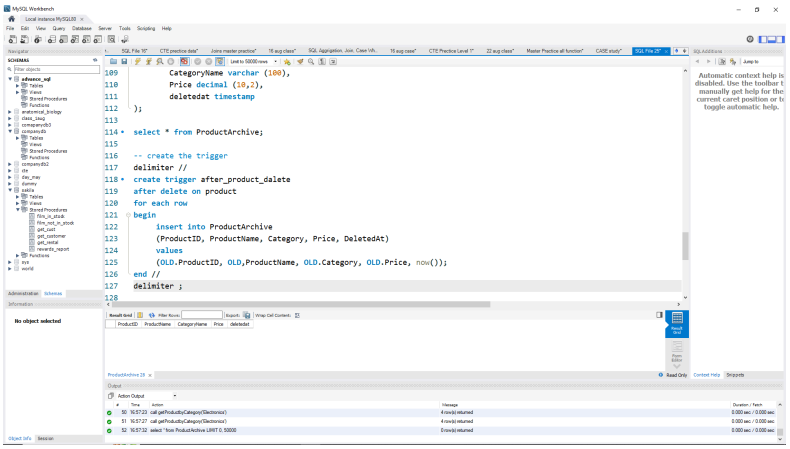
call getProductbyCategory('Electronics');
```

Q10. Create an AFTER DELETE trigger on the Products table that archives deleted product rows into a new table ProductArchive . The archive should store ProductID, ProductName, Category, Price, and DeletedAt timestamp.

Ans:- Here, `OLD` refers to the product row before it was deleted. The trigger automatically stores the deleted product's details and deletion timestamp in `ProductArchive`.



```
/*Q10. Create an AFTER DELETE trigger on the Products table that archives deleted product
rows into a new table ProductArchive . The archive should store ProductID,
ProductName, Category, Price, and DeletedAt timestamp. */
-- create the archive table
create table ProductArchive(
        ProductID int,
        ProductName varchar(100),
        CategoryName varchar (100),
        Price decimal (10,2),
        deletedat timestamp
);
select * from ProductArchive;
```



```
        CategoryName varchar (100),
        Price decimal (10,2),
        deletedat timestamp
);

select * from ProductArchive;

-- create the trigger
delimiter //
create trigger after_product_delete
after delete on product
for each row
begin
    insert into ProductArchive
    (ProductID, ProductName, Category, Price, DeletedAt)
    values
    (OLD.ProductID, OLD,ProductName, OLD.Category, OLD.Price, now());
end //
delimiter ;
```